

De la heurística de independencia al reinforcement learning

(Notebooks: independence_gap_demo + rl_examples_demo)

Abstract

Este documento junta dos ideas:

1. **Heurística de independencia.** El greedy asume que los Z_i son independientes dado el historial H y usa el producto de las marginales $\tilde{p}_i$ para puntuar cada pool. La heurística acierta casi siempre pero a veces falla feo.
2. **Reinforcement learning para DAPTS.** Formulamos DAPTS como un MDP, resolvemos el valor óptimo con *value iteration* (idéntico al DP exacto que ya tenemos), y luego mostramos como *Q-learning tabular* se acerca a esa solución sin conocer el modelo de transiciones, solo jugando muchos episodios.

La conexión es que el greedy toma un atajo heurístico porque calcular $\mathbb{P}(r_t = 0 \mid H)$ exacto es caro; el RL puede aprender una política cercana a la óptima sin necesitar ese cálculo explícito, pero a cambio de muchos episodios. Son dos puntos de un mismo espectro "costo vs. precisión".

1 Parte 1: La heurística de independencia

1.1 ¿Que estamos comparando?

Cada individuo i está infectado ($Z_i = 1$) o no ($Z_i = 0$). Después de ver el historial H (lista de pools probados y sus resultados), el greedy mantiene las marginales posteriores

$$\tilde{p}_i = \mathbb{P}(Z_i = 1 \mid H).$$

Para decidir el siguiente pool, el greedy necesita saber cuantos infectados hay en un pool candidato t , es decir la variable aleatoria

$$r_t = |t \cap Z| = \sum_{i \in t} Z_i.$$

El greedy aproxima la distribución de r_t asumiendo que los Z_i son independientes dado H . Bajo esa suposición, r_t tiene una distribución Poisson-Binomial con parámetros $(\tilde{p}_i)_{i \in t}$. En particular:

- $\mathbb{P}(r_t = 0 \mid H) \approx \prod_{i \in t} (1 - \tilde{p}_i)$ (el caso "clear" que usa el score myopico).
- $\mathbb{P}(r_t = |t| \mid H) \approx \prod_{i \in t} \tilde{p}_i$ (el caso "todos infectados").

Problema: los tests previos introducen dependencias entre los Z_i . Por ejemplo, si antes probamos $t' \subsetneq t$ y t' salió positivo (al menos un infectado), entonces t también tiene al menos un infectado con probabilidad 1. Pero la heurística le sigue dando masa positiva al evento "ningun infectado en t ".

1.2 Ejemplo concreto

Cuatro personas con prior $p_i = 0.5$. Probamos $t' = \{0, 1\}$ y el resultado fue $r' = 1$ (exactamente uno de los dos esta infectado). Ahora miramos el pool $t = \{0, 1, 2, 3\}$.

Como al menos uno de $\{0, 1\}$ esta infectado, $r_t \geq 1$ siempre: $\mathbb{P}(r_t = 0 \mid H) = 0$. Pero las marginales posteriores de 0 y 1 son $\tilde{p}_0 = \tilde{p}_1 = 0.5$ por simetria, asi que la heuristica da

$$\prod_{i \in t} (1 - \tilde{p}_i) = 0.5^4 = 0.0625.$$

	$r_t = 0$	$r_t = 1$	$r_t = 2$	$r_t = 3$	$r_t = 4$
Exacta	0.0000	0.2500	0.5000	0.2500	0.0000
Heuristica	0.0625	0.2500	0.3750	0.2500	0.0625

Table 1: Las marginales son correctas ($\tilde{p}_i = 0.5$ cada una), pero la forma de la distribucion esta mal porque los Z_i no son independientes una vez vista la historia.

1.3 ¿Que tan frecuente es el error?

Generamos 300 instancias aleatorias con $n = 8$ personas, priors uniformes en $[0.05, 0.5]$, y $B = 3$ tests seleccionados con el greedy myopico. Para cada pool candidato de tamano 2 o 3, comparamos la distribucion exacta (por enumeracion de los 2^n mundos) contra la Poisson-Binomial de las marginales.

$ t $	pools	TV media	TV mediana	TV p95	TV max
2	8400	0.024	0.000	0.217	0.500
3	16800	0.050	0.000	0.470	0.555

Table 2: Error (distancia de variacion total) entre la heuristica y la exacta. La mediana es 0 pero la cola es gorda.

Lectura. En mas de la mitad de los pools el error es exactamente cero: la heuristica funciona bien. Esto ocurre cuando el pool no se traslapa con el historial, o cuando todo el pool ya quedo determinado por los tests previos. Pero en el top 5% de los casos el error llega hasta ~ 0.47 ; el maximo (~ 0.55) aparece cuando el historial "amarra" a un subconjunto del pool dejando las marginales individuales intermedias.

1.4 Implicacion

El score myopico es

$$\text{score}(t) = \mathbb{P}(r_t = 0 \mid H) \cdot \sum_{i \in t} u_i,$$

pero se computa con $\prod(1 - \tilde{p}_i)$. Cuando esa probabilidad esta sobreestimada (como en los peores casos), el greedy puede preferir pools que nunca van a "limpiar" nada. Podriamos reemplazar la heuristica por la probabilidad exacta (via enumeracion por conteo, o aproximada con Gibbs sobre la query conjunta), pero entonces el scoring deja de ser barato. Esa tension – atajo heuristico versus computo exacto – es justo el tema de la segunda parte.

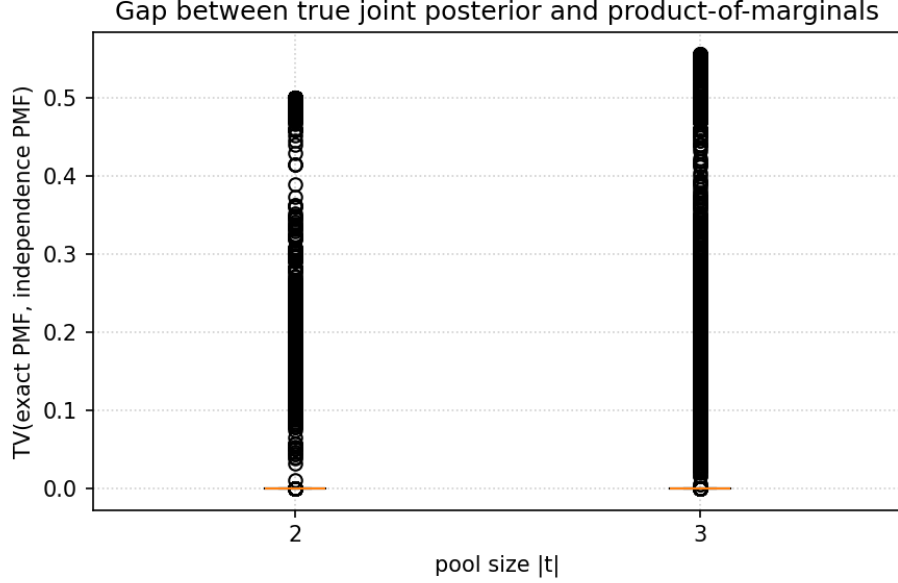


Figure 1: Error entre la PMF exacta y la heurística, agrupado por tamaño del pool. La caja está pegada a cero: la mayoría de los pools tiene error muy pequeño. Los puntos sueltos son casos raros donde el error llega casi a 0.55.

2 Parte 2: DAPTS como MDP y dos ejemplos de RL

2.1 ¿Que es un MDP?

Un *Markov Decision Process* es un modelo con:

- **Estado** s : lo que sabe el agente en un momento dado.
- **Acción** a : lo que puede hacer.
- **Transición**: dada (s, a) , hay una distribución probabilística sobre el siguiente estado s' .
- **Recompensa** $r(s, a, s')$: lo que el agente gana en esa transición.

El objetivo es maximizar la suma esperada de recompensas. La *función valor óptima* $V^*(s)$ es la recompensa total esperada si empiezas en s y juegas óptimamente. La ecuación de Bellman dice:

$$V^*(s) = \max_a \mathbb{E}[r(s, a, s') + V^*(s')].$$

2.2 DAPTS como MDP

- **Estado**: $s = (k, \text{remaining}, \text{cleared})$. Aquí k es cuántos tests ya usamos, remaining es el conjunto de perfiles de infección z consistentes con todo el historial observado, y cleared es el bitmask de individuos probados sanos.
- **Acción**: un pool $a \subseteq [n]$ con $|a| \leq G$ (el "nada" $a = \emptyset$ también es válido).
- **Transición**: observamos $r = |a \cap Z|$. Actualizamos remaining quedándonos con los z consistentes con r . Si $r = 0$, añadimos a a cleared.

- **Recompensa:** 0 en pasos intermedios. Al final (paso B), la recompensa es $\sum_{i \in \text{cleared}} u_i$.

Con esta formulacion, $V^*(s_0)$ es exactamente la utilidad esperada optima del DAPTS; coincide con lo que ya calcula `solve_optimal_dapts`.

2.3 Ejemplo 1: value iteration explicita (n=2, B=1)

Dos personas con $p = (0.3, 0.4)$, utilidades $u = (1, 1)$, un solo test, tamaño de pool ≤ 2 .

Acciones: $a \in \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$.

Calculemos $Q^*(s_0, a)$ para cada a con un paso de Bellman:

- $a = \emptyset$: nadie se limpia, $Q^* = 0$.
- $a = \{0\}$: $r = 0$ con probabilidad 0.7 (y limpia al 0 para utilidad 1); si no, $r = 1$ y nada se limpia. $Q^* = 0.7 \cdot 1 = 0.70$.
- $a = \{1\}$: analogo con $p_1 = 0.4$. $Q^* = 0.6 \cdot 1 = 0.60$.
- $a = \{0, 1\}$: $r = 0$ con probabilidad $0.7 \cdot 0.6 = 0.42$ (y limpia a ambos para utilidad 2); si no, nadie se limpia. $Q^* = 0.42 \cdot 2 = 0.84$.

La politica optima es probar a los dos juntos: $V^*(s_0) = \max = 0.84$. *Value iteration* en este caso es solo un paso de Bellman sobre el estado inicial.

2.4 Ejemplo 2: value iteration contra el DP exacto (n=3, B=2)

Tres personas con $p = (0.2, 0.3, 0.4)$, utilidades $u = (1, 2, 1.5)$, dos tests, $G = 3$.

- DP exacto (`solve_optimal_dapts`): $V^* = 2.656000$.
- Value iteration explicita sobre el MDP: $V^* = 2.656000$.
- Diferencia: $\sim 10^{-16}$ (ruido de punto flotante).

Ambos estan haciendo lo mismo: un barrido hacia atras de la ecuacion de Bellman. La diferencia es solo de vocabulario – el DP habla de "subproblemas", el MDP habla de "estados y acciones". Esto ya resuelve el problema completamente, pero *requiere saber el modelo* (las probabilidades de transicion). Eso nos lleva al siguiente paso.

2.5 Ejemplo 3: Q-learning tabular (sin modelo)

Q-learning aprende la funcion valor jugando episodios, sin necesitar conocer las probabilidades de transicion (por eso se le dice "sin modelo"). La idea por episodio:

1. Sampleamos un perfil verdadero z del prior (nada mas el agente no lo ve).
2. Empezamos en s_0 y jugamos B tests. En cada paso, con probabilidad ε probamos un pool al azar (exploracion), y con probabilidad $1 - \varepsilon$ escogemos la accion que tiene el $Q(s, a)$ mas alto hasta el momento.
3. Observamos r y pasamos al estado nuevo s' .

4. Ajustamos $Q(s, a)$ hacia el "valor que vimos": la utilidad final si s' ya es terminal, o $\max_{a'} Q(s', a')$ si todavia queda horizonte.

Para el paso de ajuste usamos un peso $\alpha = 1/(1 + N(s, a))$ (visitas al par), que es una forma estandar de garantizar que, con suficientes episodios, el aprendizaje converge.

Sobre la misma instancia ($n=3$, $B=2$) corremos 6 seeds con $\varepsilon = 0.5$ y medimos el valor de la politica aprendida despues de 500, 1000, $\dots$, 50,000 episodios.

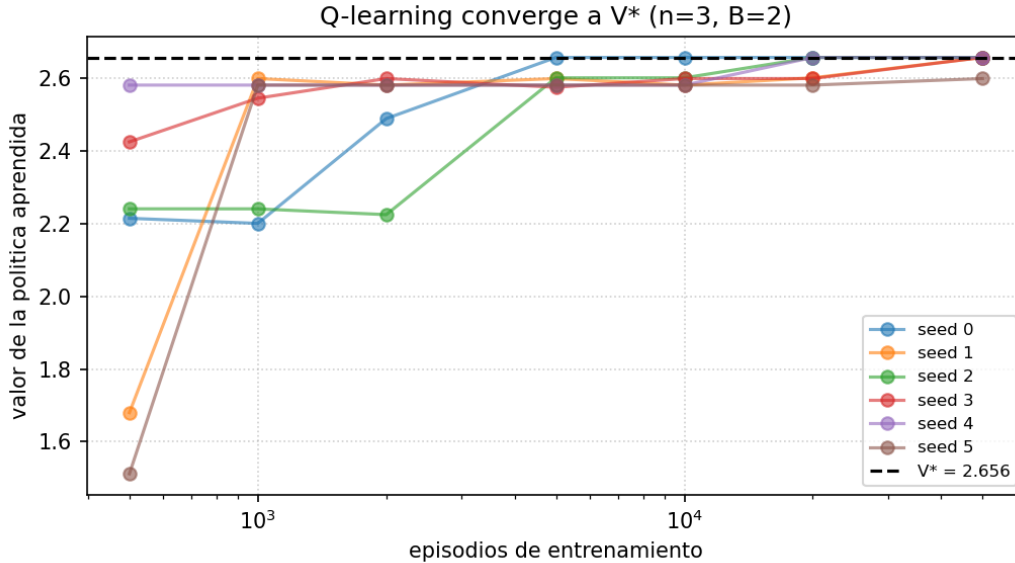


Figure 2: Q-learning tabular se acerca a $V^* = 2.656$ (linea punteada). Cada color es un seed. Con $\sim 20,000$ episodios casi todos los seeds ya llegaron al optimo; un seed se queda en 2.598 (brecha de $\sim 2\%$) incluso a 50k episodios – en el limite convergeria, pero aqui sirve como recordatorio de que Q-learning tabular no "encuentra" la solucion en un numero fijo de episodios: es un metodo iterativo que acerca, no que resuelve.

2.6 ¿Por que importa esto?

Para n chico, enumerar los 2^n perfiles es barato, asi que el DP exacto gana. Pero a medida que n crece, el numero de estados explota y no podemos hacer value iteration tabular. Ahi entra el *function approximation*: Q-learning con redes neuronales (DQN, PPO, etc.) aproxima $Q(s, a)$ con una red, generalizando entre estados parecidos. Eso es lo que ya hace `classical/r1_training/PPO_bucket_gymnasium_B*` para el modelo clasico. Adaptar esa infraestructura al modelo aumentado es el siguiente paso natural (ver §4).

3 Conexion entre las dos partes

La heuristica de independencia y el Q-learning tabular estan en extremos opuestos de un mismo eje:

Metodo	Conocimiento requerido	Precision
Heuristica $\prod \tilde{p}_i$ (greedy)	marginales $\tilde{p}_i$	aproximada
Counting + greedy	modelo completo	exacta (cara)
Gibbs + greedy	modelo completo	aproximada (escalable)
Value iteration / DP	modelo completo	optima
Q-learning tabular	solo interaccion	en el limite, optima
Q-learning + red neuronal	solo interaccion	aproximada (escala)

La heuristica es barata pero se equivoca cuando la historia correla a los miembros del pool. El DP/VI es exacto pero caro. El Q-learning tabular es un puente conceptual: muestra que, si pudieramos interactuar muchas veces con el mundo, aprenderiamos la politica optima aun sin conocer las probabilidades de transicion. Y Q-learning con redes es lo que permite escalar a n grande aceptando aproximacion controlada.

4 Siguiendo paso

Direcciones concretas:

- **Greedy con scoring exacto:** reemplazar $\prod(1-\tilde{p}_i)$ por $\mathbb{P}(r_t = 0 \mid H)$ calculado via counting o Gibbs con query conjunta, y medir si reduce la brecha contra el optimo.
- **Adaptar PPO a augmented:** reescribir el environment de gymnasium con resultado $r = |t \cap Z|$ y actualizacion bayesiana por conteo, luego entrenar PPO sobre n grande.

5 Como reproducir

```
python augmented/independence_gap_demo.py
python augmented/rl_examples_demo.py
```

Los tests unitarios estan en `augmented/tests.py` (prefijos `test_exact_pool_pmf_*`, `test_vi_*`, `test_q_learning_*`). La logica esta en `augmented/independence_gap.py` y `augmented/rl_examples.py`.